Mercury Project

Design Document

Architecture, components, message flow, authenticated data, and major design decisions.

Course	CSE499 Senior Project
Author	Matthew D. Barker
Version	RC 1.2 (1.2.0-rc.1)
Date	July 2026
Use	Academic review and grading only

Document Purpose

This document describes the Mercury RC 1.2 architecture and how its major components work together. It focuses on separation of responsibility, secure data flow, extensibility, testability, and defensive design.

Architecture Objective

Mercury protects payloads above the transport layer. Applications provide payloads and cryptographic context. Crypto providers create and authenticate SecureEnvelope instances. Codecs serialize protected envelopes. Transports move frames. Mercury Core coordinates validation, replay rejection, logging, and plaintext release decisions.

High-Level Architecture

```
Application
   |
   v
IMercuryClient
   |
   v
ICryptoProvider (SealAsync)
   |
   v
SecureEnvelope
   |
   v
IEnvelopeCodec
   |
   v
ITransport
   |
   v
Receiving IMercuryClient
   |
   v
Application
```

This separation prevents the transport and codec from becoming security decision points. It also allows new crypto providers, codecs, transports, replay protectors, loggers, and application hosts to be added without rewriting the core pipeline.

Solution Structure

Project	Responsibility
Mercury.Abstractions	Framework contracts, primitives, envelope interfaces, crypto interfaces, replay interface, transport interface, logging interface, results, and shared authenticated-data helpers.
Mercury.Core	Mercury client, dependency coordination, factories, envelope services, codecs, replay implementation, validation, and pipeline behavior.
Mercury.Provider.AesGcm	AES-GCM authenticated-encryption provider.
Mercury.Provider.ChaCha20	ChaCha20-Poly1305 authenticated-encryption provider.
Mercury.Transport.InMemory	In-memory transport for deterministic tests and demonstrations.
Mercury.Transport.Tcp	Framed TCP transport with complete-frame delivery and maximum frame guard rails.
Mercury.Transport.FileDrop	File-based transport plugin using filesystem publication and claiming behavior.

Project	Responsibility
Mercury.Demo.Console	Cross-platform console host for provider, codec, transport, text, and chunked file scenarios.
Mercury.Demo.WinForms	Windows graphical demonstration with configuration, secure-envelope display, logging, and controlled attack scenarios.
Mercury.Tests	xUnit unit, contract, pipeline, transport, codec, replay, easy-setup, and integration tests.

Core Contracts and Data Model

Contract or Model	Purpose	Important Members
IMercuryClient	Application-facing facade for protected send and receive operations.	SendAsync, ReceiveAsync
IMercuryClientDependencies	Provides explicit client identity and configured plugins to the core client.	ClientId, CryptoProvider, EnvelopeCodec, Transport, ReplayProtector, Logger
ICryptoContext	Defines sender and recipient key context for a protected exchange.	SenderKeyId, RecipientKeyId, IsEmpty
ISealRequest	Input to provider protection.	CryptoContext, Payload, HeaderMeta, FooterMeta
IOpenRequest	Input to provider authentication and decryption.	SecureEnvelope
ISecureEnvelope	Protected container passed between provider, codec, and transport.	Version, Header, Payload, Footer
IEnvelopeHeader	Authenticated envelope context.	EnvelopeId, Timestamp, SenderKeyId, RecipientKeyId, Encryption, Signature, ReplayToken, Meta
IEnvelopeFooter	Authenticated trailing metadata.	Meta
ICryptoProvider	Creates and authenticates SecureEnvelope values.	Name, SealAsync, OpenAsync
IEnvelopeCodec	Serializes and deserializes SecureEnvelope values.	Encode, Decode
ITransport	Sends and receives complete protected frames.	IsConnected, SendAsync, ReceiveAsync
IReplayProtector	Accepts new authenticated replay context and rejects duplicates.	TryAcceptAsync
IMercuryResult / FailureReason	Reports controlled success or failure without plaintext leakage.	Success, Payload, ValidatedEnvelope, FailureReason, Message

Send Pipeline

1. The application calls SendAsync with a non-empty payload and ICryptoContext.
2. Mercury validates cancellation, payload, client configuration, and context.
3. Mercury builds an ISealRequest using the application payload and optional metadata.
4. The selected ICryptoProvider creates the header and footer, builds canonical authenticated envelope data, generates a nonce and replay token, and protects the payload.
5. The provider returns a validated SecureEnvelope.
6. The selected IEnvelopeCodec serializes the SecureEnvelope into a transportable frame.
7. The configured ITransport sends only the encoded protected frame.

Receive Pipeline

8. The transport returns one complete protected frame.
9. The codec structurally decodes the frame into a SecureEnvelope.
10. Mercury validates required envelope structure and intended recipient context.

11. The selected crypto provider reconstructs canonical authenticated data and authenticates and decrypts the protected payload.
12. Only after cryptographic validation succeeds does Mercury call the replay protector with the authenticated header.
13. If replay protection accepts the sender/token combination, Mercury returns the plaintext payload.
14. Decode, authentication, recipient, replay, provider, or transport failures return controlled failure results without plaintext.

Cryptographic Provider Design

Mercury RC 1.2 includes operational AES-GCM and ChaCha20-Poly1305 providers. Both providers use the same ICryptoProvider contract and the same canonical authenticated envelope data. AES-CCM remains pending and is not claimed as completed support.

Each completed provider resolves a symmetric key by KeyId, validates the algorithm-required key length, generates fresh per-message cryptographic material, performs authenticated encryption, and packages the nonce, authentication tag, and ciphertext into the protected payload format.

Canonical Authenticated Envelope Data

AuthenticatedEnvelopeData produces a deterministic, length-prefixed representation of security-relevant envelope fields. Metadata entries are sorted using ordinal comparison so both endpoints authenticate the same byte sequence regardless of dictionary enumeration order.

- Authenticated-data format marker and format version
- Framework major and minor version
- Envelope identifier
- Timestamp in Unix milliseconds
- Sender and recipient key identifiers
- Encryption and signature algorithm identifiers
- Replay token
- Header metadata sorted by ordinal key
- Footer metadata sorted by ordinal key

Changing any authenticated field causes provider authentication to fail. This design replaced the earlier delimiter-based approach and permits unambiguous field encoding without relying on delimiter restrictions.

Key and Identifier Design

- KeyId and AlgorithmId are value types with consistent default and empty behavior.
- Equality and hashing use ordinal, case-sensitive comparison.
- SymmetricKeyProviderDictionary rejects empty KeyId values, missing keys, and null or empty key material.
- Key arrays are defensively copied when stored and when returned.
- Algorithm-specific key-size validation remains inside each crypto provider.

Envelope Codec Design

Mercury provides Binary and JSON codecs. A codec is responsible for structural serialization and deserialization only. It does not provide cryptographic integrity. Security integrity is enforced by the selected provider after decoding.

- Binary codec uses a versioned magic value, explicit big-endian length prefixes, envelope fields, metadata, protected payload, and trailing-data rejection.
- JSON codec uses named fields and Base64 for binary values to support demonstrations and diagnostics.

- Wrong-codec, malformed, truncated, unsupported-version, and invalid Base64 inputs are rejected before plaintext release.
- Codecs preserve the complete SecureEnvelope across encode and decode round trips.

Transport Design

Transports are unaware of plaintext and cryptographic decisions. They send and receive complete protected frames through the ITransport boundary.

Transport	Design
In-memory	Deterministic frame delivery for tests and demonstrations, with defensive copying and maximum frame-size enforcement.
TCP	Four-byte big-endian length prefix followed by one complete frame, separate send/receive synchronization, cancellation, and maximum frame-size enforcement.
FileDrop	Publishes complete frames through filesystem files and claim/processing behavior. It is included as a plugin but retains more operational recovery work than the primary in-memory and TCP paths.

Replay Protection Design

InMemoryReplayProtector identifies a replay by the exact sender KeyId and replay-token bytes. It uses a configurable replay window and maximum active-entry count. Expired entries are removed before capacity checks. When capacity is reached, the protector fails closed by rejecting additional records. Process restart clears the in-memory state.

Logging and Failure Handling

- IMercuryLogger allows hosts to observe pipeline behavior without coupling Mercury Core to a logging framework.
- Caller errors such as empty payloads or invalid required arguments throw appropriate exceptions.
- Receive-side decode, authentication, replay, provider, and transport failures return controlled results where possible.
- OperationCanceledException is allowed to propagate.
- Failure results do not include plaintext payload data.
- Logs and demo output must not expose key material, plaintext after failure, authentication tags, or other secrets.

Demo Architecture

Mercury.Demo.Console provides a cross-platform host. Mercury.Demo.WinForms provides the primary graphical demonstration and uses designer-placed custom controls, a controller-based backend, configurable provider/transport/codec selection, event logging, secure-envelope inspection, chunked file sending, and controlled replay, tamper, and wrong-key scenarios.

The TCP attack simulator is isolated in the demo project. It is not part of Mercury Core or Mercury.Transport.Tcp. Its purpose is to alter or replay complete protected frames so the demo can show framework rejection behavior.

Major Design Decisions

Decision	Reason
Security above transport	Keeps protection independent of network or delivery mechanism.
Explicit client KeyId	Allows the receiving client to validate intended recipient context.

Decision	Reason
Plugin interfaces	Makes crypto, codecs, transports, replay protection, and logging swappable and testable.
SecureEnvelope boundary	Ensures only protected data crosses the transport boundary.
Canonical authenticated data	Binds security-relevant envelope fields without ambiguous string concatenation.
Replay after authentication	Prevents unauthenticated metadata from polluting replay state.
Bounded replay storage	Prevents unlimited in-memory growth during the configured replay window.
ReadOnlyMemory payload abstraction	Supports text, binary, and file scenarios while preventing accidental mutation.
Controlled results	Supports safe failure without releasing plaintext.
Separate console and WinForms hosts	Demonstrates portability without forcing the framework to depend on a UI technology.
No certification claims	Keeps the academic submission honest and reviewable.

Extensibility Points

- Additional ICryptoProvider implementations after complete contract and pipeline testing.
- Additional ITransport plugins such as WebSocket, named pipes, serial, HTTP relay, or queue transports.
- Persistent or distributed IReplayProtector implementations.
- Additional envelope codecs with explicit versioning and structural limits.
- Production key providers backed by secure storage or hardware security modules.
- Signature-based authentication and certificate-backed identity.
- Additional application hosts without changes to Mercury Core.